

User Manual

SIP++

Semiconductor Ion Pulse Simulator (C++ Engine)



Saeed Boorboor

Code Manifesto

SIP++ is a high-performance computational framework designed to simulate the transient charge induction and pulse shape evolution in semiconductor detectors specifically under **Heavy Ion** irradiation. It bridges the gap between raw physical interaction (energy deposition) and the final digitized electronic signal.

Core Features

- **Heavy Ion Dynamics:** Modeling of high-density charge tracks and plasma column effects.
- **Adaptive nested mesh:** Optimize computation performance by fining mesh around the plasma.
- **Transient Solver:** Calculation of charge carrier transport (electrons and holes).
- **C++ Efficiency:** Optimized for speed, allowing for large-scale Monte Carlo integration.

Dependencies

- ROOT

The CERN-developed ROOT library is utilized for plotting, visualization, and file management functionalities.

- Garfield++

Garfield++ is required to obtain physical data related to charge carrier transport, including parameters such as mobility.

- GEANT4

GEANT4 is employed to simulate primary ion tracks and to compute the spatial distribution of energy deposited within the detector's active region.

1- How SIP++ works

A simplified schematic of SIP++ workflow is presented in Figure 1. The framework consists of two applications, SIPEventGenerator and SIPMain, which operate sequentially.

The simulation workflow begins with SIPEventGenerator, which produces a detailed energy deposition profile within the detector's sensitive volume. This application is implemented using the GEANT4 toolkit and simulates the transport of the primary particle and the associated delta rays through the silicon material. The output of SIPEventGenerator, including detailed information about particle steps, is stored in a ROOT file with the default name G4Events.root.

The main simulation tasks are performed by SIPMain. In the initial stage, the user-selected event is converted into electron–hole (e–h) pairs according to the energy deposition profile. After charge generation, SIPMain enters a time-dependent simulation loop that iteratively models charge carrier transport and ultimately produces an electric pulse for each event.

At the beginning of each time step, the adaptive computational mesh is updated according to the current charge distribution. This nested meshing approach provides high spatial resolution in regions with high carrier density, where plasma effects are most significant, while maintaining computational efficiency in other regions. The electric field is then calculated by directly solving Coulomb's law on the updated mesh. Subsequently, drift and diffusion models are applied to update the positions of the charge carriers.

The time loop continues until either a predefined simulation time is reached or all charge carriers have been collected at the electrodes or have recombined. During the simulation, the state of SIPMain, including all data required to resume the simulation and for subsequent post-processing, is periodically written at user-defined intervals to files named Continue_(time).root. In addition, optional graphical outputs can be enabled to visualize the positions of the charge carriers, the electric field intensity, and the resulting current signals.

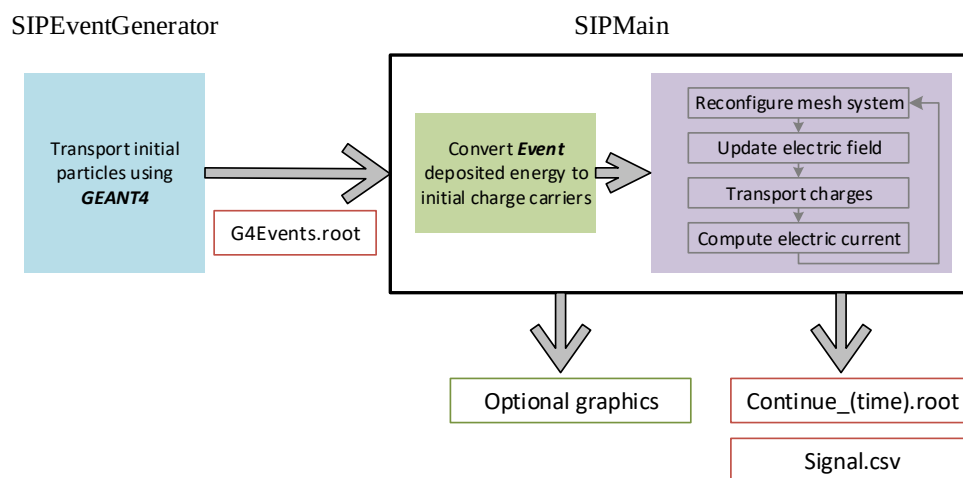


Figure 1: Schematic of SIP++ workflow.

2- How to install

The applications were compiled using GCC 13.3 on Ubuntu 24, and the resulting executable binaries are available on the GitHub page at <https://github.com/boorboor-saeed/SIP/bin>. Download the bin directory and put it at a suitable place in your machine. Add bin directory to system path.

To run these binaries, the required libraries—GEANT4, Garfield++, and ROOT—must be installed in advance. The dependency structure of the binaries is illustrated in Figure 2.

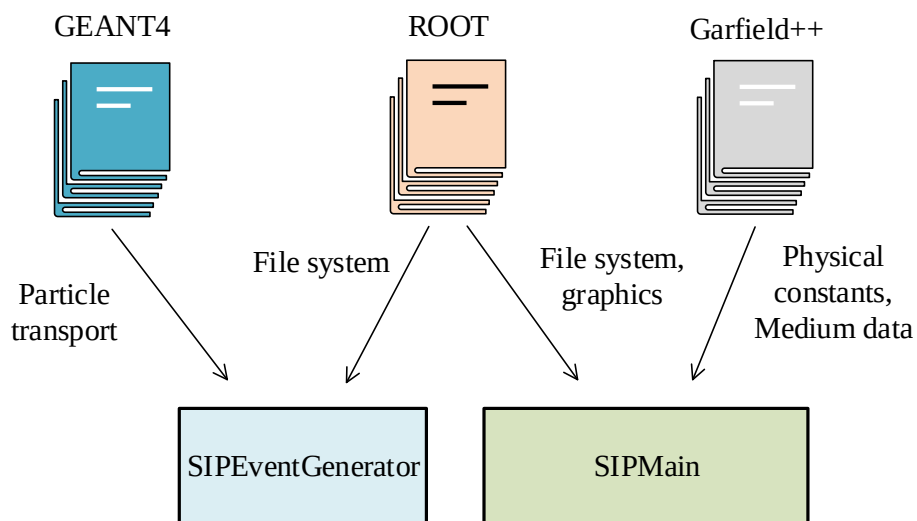


Figure 2: binary dependencies.

3- How to run

To run the simulation, the two applications SIPEventGenerator and SIPMain must be executed consecutively.

Phase one

The first stage of the workflow, illustrated in Figure 3, involves SIPEventGenerator, which reads an input file—by default named EventGenerator.mac. This file specifies the detector setup and the characteristics of the incident particle using the standard G4Messenger command format. A complete list of available commands and their parameters is provided in Appendix 1.

The output from SIPEventGenerator, containing detailed particle step information, is saved in a ROOT file with the default name G4Events.root.

Two execution modes are available for SIPEventGenerator:

- Run without arguments

```
>> ./SIPEventGenerator
```

- Run with a user-specified input file as the single argument

```
>> ./SIPEventGenerator PathToInputFile/InputFile.mac
```

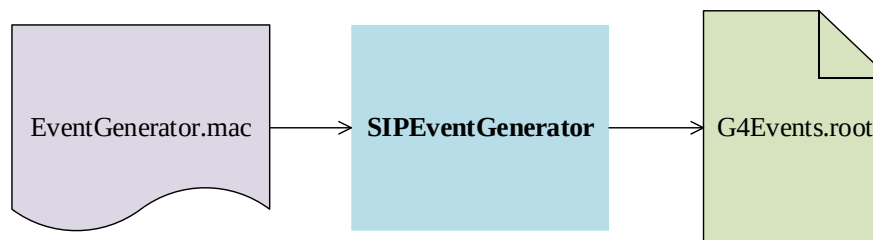


Figure 3: Generate events as the first step of the simulation workflow.

Phase two

The second stage involves running SIPMain. As illustrated in Figure 4, this application uses two files—SIPMain.mac and G4Events.root—to begin a new charge transport simulation. The SIPMain.mac file defines the detector properties, biasing conditions, the path to the G4Events file, the run time, and other configuration parameters. The details of the input commands in SIPMain.mac are provided in Appendix 2.

The charge transport process can also be resumed from a specific simulation time. As shown in Figure 5, to continue a simulation from a given time, the application requires a Continue_(time).root file as input. In addition, SIPMain can display the contents of a continue file.

Four modes of operation is considered for SIPMain:

- Start a fresh run without arguments

```
>> ./SIPMain
```

- Start a fresh run with a user specified input file

```
>> ./SIPMain PathToInputFile/InputFile.mac
```

- Continue a run from a specified time

```
>> ./SIPMain Continue PathToContinueFile/ContinueFile.root
```

- Display the contents of a continue file

```
>> ./SIPMain Draw PathToContinueFile/ContinueFile.root
```

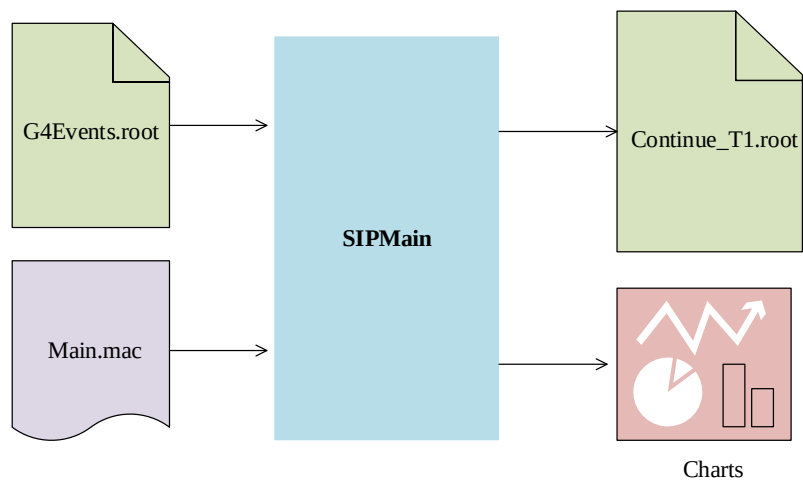


Figure 4: Start charge transport as the second step of the simulation workflow.

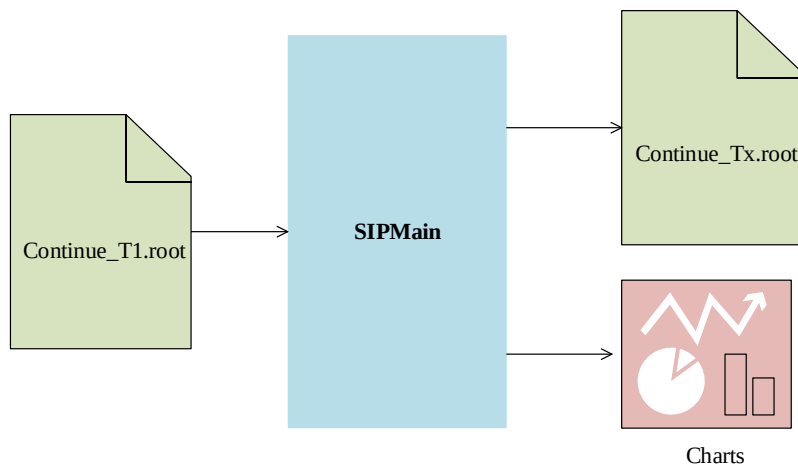


Figure 5: Resume charge transport simulation from a specified time.

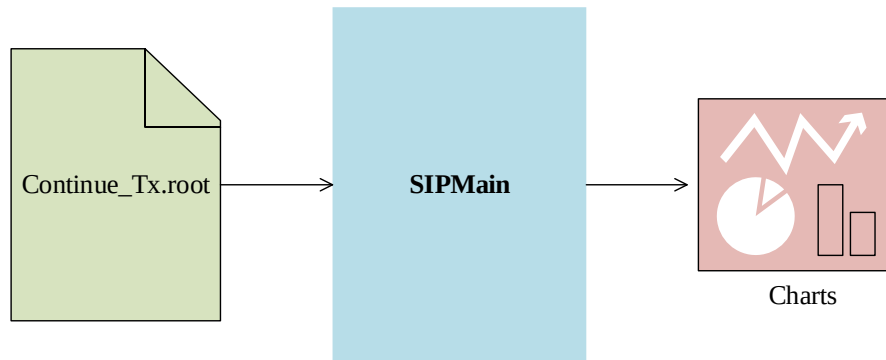


Figure 6: Draw the content of a continue file for post-processing.

Example:

To demonstrate how to use **SIP++** in Ubuntu, consider running the following example.

Download the *Benchmarks* directory from <https://github.com/boorboor-saeed/SIP/Benchmarks> and place it in a suitable location on your local machine. This directory contains several benchmark cases that compare the simulation results with experimental measurements.

- Navigate to `Benchmarks/DetA/Alpha`.
- Execute **SIPEventGenerator** using the default input file:

```
>> ./SIPEventGenerator
```

If the execution is successful, **SIPEventGenerator** will generate a file named `G4Events.root`, which contains detailed information about the alpha particle track inside the detector.

- Execute **SIPMain** using the default input file:

```
>> ./SIPMain
```

This command starts the charge transport simulation. Three canvases will also open to display the electric field distribution, carrier positions, and the resulting electric current signal.

4- Appendixes

Appendix 1

Commands and associated parameters used in **SIPEventGenerator**.

- Define an element

```
/Material/DefineElement ElementName AtomicNumber AtomicMass(g/mol)
```

Example: define element of gold

```
/Material/DefineElement Au 79 196.977
```

- Define a material

```
/Material/DefineMaterial MaterialName Density(g/cm3) NumberOfConstituents
```

Example: define material of gold

```
/Material/DefineMaterial Gold 19.3 1
```

- Define detector

```
/Geometry/CreateDetector Radius AnodeThickness CathodeThickness  
ActiveThickness AnodeMaterial CathodeMaterial ActiveMaterial
```

- All dimensions are in millimeter.

Example: define a planar detector

```
/Geometry/CreateDetector 5 100e-6 100e-6 0.5 Gold Silicon Silicon
```

Appendix 2

Commands and associated parameters used in **SIPMain**.

- Detector features

Command	Parameter type	Unit	Description
Thickness	double	μm	Thickness of active region
BiasVoltage	double	volt	Anode to cathode voltage
DepletionVoltage	double	volt	Minimum voltage to reach depletion
Temperature	double	K	Temperature of lattice

Example: consider a detector with thickness of 330 micro meter, biased at -38v, depletion voltage of -6v, and temperature of 293 K.

```
Thickness 330.  
BiasVoltage -38.5  
DepletionVoltage -6  
Temperature 293.
```

- Activate/Deactivate modeling

Command	Parameter type	Acceptable values	Description
DynamicField	bool	ON/OFF	on or off dynamic field calculation
ConcentrationMobility	bool	ON/OFF	on or off concentration mobility dependency

Example: activate dynamic field and mobility dependency on charge concentration

```
DynamicField ON  
ConcentrationMobility ON
```

- Set mesh level

Command	Parameter type	Acceptable values	Description
MeshDepth	int	1 - 5	Determine number of nesting levels

Example: consider a nested mesh with 3 levels

```
MeshDepth 3
```

- Set address of input events

Command	Parameter type	Acceptable values	Description
G4EventsFile	string	Relative or absolute address	Address of ROOT file containing generated event by SIPEventGenerator

Example: consider a events in file named G4Events_Alpha5.5MeV.root in the same directory

G4EventsFile G4Events_Alpha5.5MeV.root

- Select event to process

Command	Parameter type	Acceptable values	Description
EventID	int	Maximum number of events in G4Events file	Select the event to be converted to initial charges

Example: process the first event

EventID 0

- Select supercharge magnitude

Command	Parameter type	Acceptable values	Description
SuperChargeScale	int	1 - 500	Select supercharge magnitude

Example: consider super charge with magnitude of 10

SuperChargeScale 10

- Limit view pane limit

Command	Xmin	Xmax	Zmin	Zmax	Unit	Description
ViewLimit	double	double	double	double	μm	Select view pane limit for charge and field representation

Example: consider view X limit between -30 to 30 μm and Z limit between 0 to 330 μm

ViewLimit -30 30 0 330.

- Select refresh view time

Command	Parameter type	Acceptable values	Description
RefreshViewTime	int	1 - 500	Interval between iteration number to refresh view panes

Example: refresh view pane every 10 iteration step

```
RefreshViewTime 10
```

- Select save output directory

Command	Parameter type	Acceptable values	Description
SaveOutputDir	string	Relative or absolute address	The directory to keep simulation results

Example: consider output directory next to directory containing executable

```
SaveOutputDir ../OutputDir
```

- Select save time

Command	Parameter type	Acceptable values	Description
SaveContinueTime	int	1 - 500	Interval between iteration number to save outputs

Example: save output every 10 iteration step

```
SaveContinueTime 10
```